



Universidad Internacional del Ecuador

**Selección del Programa a desarrollar / Generación de Diagramas
funcionales y Arquitectura de Software**

Actividad académica – Aprendizaje autónomo 1

Estudiante: Erick Pries

Carrera: Ingeniería en Ciber Seguridad

Materia: Lógica de programación 1

Docente: Mgs. Mónica Patricia Salazar Tapia

Fecha: 31 de mayo de 2026

1. Introducción

El desarrollo de software no debe iniciar directamente con la programación, sino con una etapa previa de análisis y diseño. Esta fase permite comprender claramente el problema que se desea resolver, identificar los requerimientos principales del sistema y representar visualmente cómo funcionará la solución antes de construirla. De esta manera, se reducen errores, se mejora la organización del proyecto y se facilita la comunicación de la idea.

En esta actividad se selecciona un programa a desarrollar entre varias alternativas propuestas, tales como el juego del ahorcado, piedra, papel o tijera, Atari Pong, juego de la serpiente, generador seguro de contraseñas y juego “Adivina el número”. La opción elegida para este trabajo es el **Juego de la Serpiente**, debido a que permite aplicar conceptos básicos de programación, estructuras de control, interacción con el usuario, manejo de eventos, actualización de estados, colisiones y diseño modular.

El Juego de la Serpiente consiste en controlar una serpiente que se mueve dentro de un área limitada. El jugador debe dirigirla hacia la comida que aparece en posiciones aleatorias. Cada vez que la serpiente come, aumenta su tamaño y se incrementa el puntaje. La partida termina cuando la serpiente choca contra los límites del escenario o contra su propio cuerpo.

Para diseñar esta solución se utilizarán diagramas funcionales y diagramas de arquitectura. Los diagramas funcionales permiten representar qué hará el sistema y cómo interactuará el usuario con él. Por otro lado, los diagramas de arquitectura muestran la estructura general del software, sus componentes principales y la relación entre ellos.

El objetivo de este documento es presentar el análisis del problema, justificar la selección del programa, identificar sus funcionalidades principales y proponer diagramas que permitan comprender el funcionamiento y la arquitectura del sistema antes de iniciar su desarrollo.

2. Investigación sobre diagramas funcionales y de arquitectura

Antes de diseñar un software, es importante conocer los tipos de diagramas que pueden utilizarse para representar sus funcionalidades y su estructura. Los diagramas ayudan a transformar una idea general en una propuesta más organizada, visual y comprensible.

2.1. Diagramas funcionales

Los diagramas funcionales representan el comportamiento del sistema, es decir, muestran qué acciones realiza el software, qué decisiones toma, qué procesos ejecuta y cómo interactúa con los usuarios o actores externos.

Estos diagramas son útiles porque permiten identificar los requerimientos funcionales antes de programar. Un requerimiento funcional describe una acción o servicio que el sistema debe realizar. Por ejemplo, en el Juego de la Serpiente, algunos requerimientos funcionales son: iniciar partida, mover la serpiente, generar comida, detectar colisiones, actualizar el puntaje y reiniciar el juego.

Entre los principales diagramas funcionales se encuentran:

2.1.1. Diagrama de casos de uso

El diagrama de casos de uso pertenece al Lenguaje Unificado de Modelado, conocido como UML.¹ Este tipo de diagrama muestra la relación entre los actores externos y las funcionalidades principales del sistema.

Un actor representa a una persona, usuario u otro sistema que interactúa con el software. Un caso de uso representa una acción o funcionalidad que el sistema ofrece.

En el Juego de la Serpiente, el actor principal es el **Jugador**. Este actor puede iniciar una partida, controlar la dirección de la serpiente, visualizar el puntaje, perder la partida y reiniciar el juego.

El diagrama de casos de uso es útil en este proyecto porque permite responder preguntas como:

- ¿Quién usa el sistema?
- ¿Qué acciones puede realizar el usuario?
- ¿Qué funcionalidades debe ofrecer el programa?
- ¿Qué procesos internos se activan cuando el usuario interactúa con el juego?

Este tipo de diagrama no describe en detalle cómo se programa cada funcionalidad, sino qué funcionalidades existirán desde la perspectiva del usuario.

2.1.2. Diagrama de flujo

El diagrama de flujo representa la secuencia lógica de un proceso. Utiliza símbolos como óvalos, rectángulos, rombos y flechas para mostrar el inicio, las acciones, las decisiones y el final de un proceso.²

En programación, los diagramas de flujo son especialmente útiles para representar algoritmos. En el caso del Juego de la Serpiente, permiten mostrar el ciclo principal del juego: iniciar partida, mover la serpiente, verificar si comió comida, actualizar puntaje, comprobar colisiones y finalizar o continuar la partida.

Este tipo de diagrama ayuda a entender la lógica general del sistema antes de escribir código. También permite identificar decisiones importantes, como:

- ¿La serpiente comió comida?
- ¿La serpiente chocó contra una pared?
- ¿La serpiente chocó contra su propio cuerpo?
- ¿El jugador desea reiniciar la partida?

2.1.3. Diagrama de actividades

El diagrama de actividades también pertenece a UML y se utiliza para representar flujos de trabajo. Es similar al diagrama de flujo, pero suele utilizarse en contextos más formales de modelado de software.

Este diagrama puede mostrar actividades paralelas, condiciones, bifurcaciones y procesos internos. Para un juego sencillo como el de la serpiente, un diagrama de flujo puede ser suficiente, aunque el diagrama de actividades también podría aplicarse para representar el comportamiento del juego.

2.1.4. Diagrama de estados

El diagrama de estados representa los diferentes estados por los que puede pasar un sistema o un objeto. En el Juego de la Serpiente, el sistema puede encontrarse en varios estados:

- Pantalla inicial.
- Partida en ejecución.
- Partida pausada, si se decide incluir pausa.
- Fin de partida.
- Reinicio de partida.

Este diagrama sería útil si se desea representar cómo el juego cambia de un estado a otro. Por ejemplo, al presionar “Iniciar”, el juego pasa de la pantalla inicial al estado de ejecución. Si ocurre una colisión, pasa al estado de fin de partida.

2.1.5. Diagrama de secuencia

El diagrama de secuencia muestra la interacción entre objetos o componentes a lo largo del tiempo. Es útil para representar el orden en que se comunican los elementos del sistema.

En el Juego de la Serpiente, podría utilizarse para mostrar cómo el controlador principal recibe una entrada del teclado, actualiza la dirección de la serpiente, mueve el cuerpo, verifica colisiones, actualiza el puntaje y redibuja la pantalla.

Aunque es un diagrama útil, para esta actividad se seleccionan principalmente el diagrama de casos de uso y el diagrama de flujo, ya que son más adecuados para explicar las funcionalidades generales del sistema.

2.2. Diagramas de arquitectura de software

Los diagramas de arquitectura de software representan la estructura general de un sistema. Mientras los diagramas funcionales se enfocan en lo que hace el sistema, los diagramas de arquitectura explican cómo se organiza internamente.³

La arquitectura de software permite identificar los componentes principales del programa, sus responsabilidades y la forma en que se comunican entre sí. Esta etapa es importante porque ayuda a construir un sistema más ordenado, mantenible y fácil de modificar.

En el caso del Juego de la Serpiente, aunque se trata de un programa pequeño, también es conveniente separar responsabilidades. Por ejemplo, no sería recomendable mezclar toda la lógica en una sola parte del código. Es mejor dividir el sistema en módulos como entrada de teclado, lógica de movimiento, comida, colisiones, puntaje e interfaz gráfica.

Entre los diagramas de arquitectura más comunes se encuentran:

2.2.1. Diagrama de componentes

El diagrama de componentes muestra las partes principales del sistema y cómo se relacionan. Cada componente representa una sección del software con una responsabilidad específica.

Para el Juego de la Serpiente, los componentes principales podrían ser:

- Interfaz gráfica.
- Controlador principal del juego.
- Módulo de entrada del teclado.
- Módulo de lógica de la serpiente.
- Módulo de comida.
- Módulo de colisiones.
- Módulo de puntaje.
- Módulo de estado del juego.

Este tipo de diagrama es adecuado para esta actividad porque permite explicar de forma clara cómo estará dividido el programa.

2.2.2. Diagrama de capas

El diagrama de capas organiza el software en niveles. Por ejemplo, un sistema puede dividirse en capa de presentación, capa de lógica y capa de datos.

En el Juego de la Serpiente se puede plantear una arquitectura sencilla de tres capas:

- **Capa de presentación:** muestra el juego en pantalla.
- **Capa de lógica:** controla el movimiento, las colisiones, la comida y el puntaje.
- **Capa de estado:** conserva la información actual del juego, como posición de la serpiente, puntaje y estado de la partida.

Aunque el juego no requiere una base de datos, sí necesita almacenar temporalmente datos durante la ejecución, como la posición de la serpiente, la ubicación de la comida y el puntaje.

2.2.3. Modelo C4

El modelo C4⁴ es una forma de representar arquitectura de software mediante diferentes niveles de detalle. Sus niveles más conocidos son:

- **Contexto:** muestra el sistema completo y sus usuarios.
- **Contenedores:** muestra las aplicaciones o partes principales que componen el sistema.
- **Componentes:** muestra los módulos internos de cada contenedor.
- **Código:** muestra clases, funciones o elementos más detallados.

Para este trabajo no es necesario aplicar todos los niveles del modelo C4, pero sí puede tomarse como referencia para estructurar la arquitectura del juego desde una vista general hacia una vista más detallada.

En este caso, el contexto sería el jugador interactuando con el Juego de la Serpiente. El contenedor sería la aplicación del juego. Los componentes serían la interfaz gráfica, el controlador, la lógica de movimiento, las colisiones, la comida y el puntaje.

2.2.4. Diagrama de despliegue

El diagrama de despliegue muestra dónde se ejecuta el sistema, es decir, en qué dispositivos, servidores o entornos funciona el software.

Para el Juego de la Serpiente, el despliegue sería sencillo: el programa se ejecutaría localmente en una computadora del usuario. No requiere conexión a internet, servidor externo ni base de datos.

2.3. Diagramas seleccionados para este proyecto

Para esta actividad se seleccionan los siguientes diagramas:

- **Diagrama de casos de uso:** para representar las funcionalidades desde el punto de vista del jugador.
- **Diagrama de flujo funcional:** para explicar la lógica principal del juego.
- **Diagrama de arquitectura por componentes:** para representar cómo se organizará internamente el sistema.

Estos diagramas son adecuados porque permiten comprender el juego antes de programarlo. Además, facilitan la explicación en el video y cumplen con el objetivo de diseñar la solución antes del desarrollo.

3. Selección del programa a desarrollar

Entre las opciones propuestas, se selecciona el **Juego de la Serpiente**.

Esta opción fue elegida porque permite desarrollar un programa interactivo, visual y fácil de comprender. A diferencia de programas más simples, como piedra, papel o tijera o adivina el número, el Juego de la Serpiente incorpora movimiento continuo, control por teclado, generación aleatoria de elementos, crecimiento del personaje, detección de colisiones y sistema de puntaje.

Además, este juego permite aplicar varios conceptos fundamentales de programación, tales como:

- Variables.
- Condicionales.
- Ciclos.
- Listas o arreglos.
- Eventos del teclado.
- Funciones.
- Coordenadas en pantalla.
- Generación aleatoria.
- Validación de condiciones.
- Organización modular del código.

El Juego de la Serpiente también es una buena opción porque su funcionamiento es conocido por muchas personas, lo que facilita explicar el objetivo del programa y representar su lógica mediante diagramas.

El programa no requiere una conexión a internet ni una base de datos. Puede ejecutarse de forma local en una computadora. Esto permite enfocar el análisis en la lógica del juego y en la interacción entre el usuario y el sistema.

4. Análisis del problema

El problema consiste en diseñar un juego en el que el usuario pueda controlar una serpiente dentro de un espacio limitado. La serpiente debe moverse constantemente y el jugador debe cambiar su dirección para alcanzar la comida que aparece en la pantalla.

Cada vez que la serpiente come, debe ocurrir lo siguiente:

- El puntaje aumenta.
- La serpiente crece.

- Se genera una nueva comida en una posición aleatoria.
- El juego continúa.

Sin embargo, el jugador debe evitar que la serpiente choque contra los límites del área de juego o contra su propio cuerpo. Si ocurre una de estas colisiones, la partida termina y se muestra un mensaje de fin de juego.

El sistema debe resolver varios aspectos importantes:

4.1. Movimiento de la serpiente

La serpiente debe moverse automáticamente en una dirección determinada. El jugador no controla cada paso individual, sino que cambia la dirección de movimiento mediante el teclado.

Por ejemplo:

- Flecha arriba: mover hacia arriba.
- Flecha abajo: mover hacia abajo.
- Flecha izquierda: mover hacia la izquierda.
- Flecha derecha: mover hacia la derecha.

El sistema debe evitar movimientos inválidos, como cambiar directamente de derecha a izquierda cuando la serpiente se está moviendo hacia la derecha, ya que eso provocaría que choque inmediatamente consigo misma.

4.2. Representación del cuerpo de la serpiente

La serpiente puede representarse como una lista de posiciones. Cada posición corresponde a una parte del cuerpo. La primera posición representa la cabeza y las demás representan el cuerpo.

Cuando la serpiente se mueve, se agrega una nueva posición al frente de la lista y se elimina la última posición, simulando el movimiento. Si la serpiente come, no se elimina la última posición, lo que hace que crezca.

4.3. Generación de comida

La comida debe aparecer en una posición aleatoria dentro del área de juego. Sin embargo, la comida no debe generarse sobre el cuerpo de la serpiente, ya que eso causaría errores o situaciones injustas para el jugador.

Por lo tanto, el sistema debe verificar que la nueva posición de la comida esté libre.

4.4. Sistema de puntaje

El puntaje debe iniciar en cero. Cada vez que la serpiente come, el puntaje aumenta. El valor puede ser de 1 punto por comida o de 10 puntos, dependiendo del diseño del juego.

El puntaje debe mostrarse en pantalla para que el jugador conozca su avance.

4.5. Detección de colisiones

El sistema debe detectar dos tipos principales de colisión:

- Colisión con paredes.
- Colisión con el propio cuerpo.

La colisión con paredes ocurre cuando la cabeza de la serpiente sale del área permitida. La colisión con el cuerpo ocurre cuando la cabeza ocupa una posición que ya pertenece a otra parte de la serpiente.

Si se detecta una colisión, el sistema debe cambiar el estado de la partida a “finalizada”.

4.6. Fin de partida y reinicio

Cuando la partida termina, el sistema debe mostrar un mensaje de “Game Over” y el puntaje final. Además, debe permitir al jugador reiniciar la partida para comenzar nuevamente desde cero.

5. Funcionalidades del sistema

Las funcionalidades del sistema son las acciones principales que el juego debe realizar. Para este proyecto se identifican las siguientes:

5.1. Iniciar partida

El sistema debe permitir iniciar una nueva partida. Al iniciar, se crea la serpiente en una posición inicial, se genera la primera comida, se coloca el puntaje en cero y se establece el estado del juego como “en ejecución”.

5.2. Controlar dirección de la serpiente

El jugador debe poder cambiar la dirección de la serpiente utilizando el teclado. El sistema debe leer la entrada del usuario y actualizar la dirección del movimiento.

5.3. Mover serpiente

El sistema debe mover la serpiente automáticamente en intervalos de tiempo. En cada movimiento, la cabeza avanza hacia la dirección actual y el cuerpo sigue la posición anterior de la cabeza.

5.4. Generar comida

El sistema debe generar comida en una posición aleatoria válida dentro del área de juego. La posición debe estar libre y no debe coincidir con el cuerpo de la serpiente.

5.5. Detectar comida

El sistema debe verificar si la cabeza de la serpiente coincide con la posición de la comida. Si esto ocurre, se considera que la serpiente comió.

5.6. Aumentar tamaño de la serpiente

Cuando la serpiente come, su tamaño debe aumentar. Esto se logra manteniendo una parte adicional del cuerpo en lugar de eliminar la última posición durante el movimiento.

5.7. Actualizar puntaje

Cada vez que la serpiente come, el sistema debe aumentar el puntaje y actualizarlo en pantalla.

5.8. Detectar colisiones

El sistema debe verificar si la serpiente choca contra una pared o contra su propio cuerpo. Esta funcionalidad es fundamental porque define cuándo termina la partida.

5.9. Finalizar partida

Cuando ocurre una colisión, el sistema debe detener el movimiento de la serpiente y mostrar la pantalla de fin de juego.

5.10. Mostrar puntaje final

Al finalizar la partida, el sistema debe mostrar el puntaje obtenido por el jugador.

5.11. Reiniciar partida

El jugador debe poder reiniciar el juego después de perder. Al reiniciar, el sistema vuelve a crear la serpiente, reinicia el puntaje y genera una nueva comida.

5.12. Salir del juego

Opcionalmente, el sistema puede permitir cerrar el juego desde la pantalla de fin de partida o mediante una tecla específica.

6. Diagrama de casos de uso

El diagrama de casos de uso representa las funcionalidades principales desde la perspectiva del jugador.

En este sistema, el actor principal es el **Jugador**, quien interactúa directamente con el juego. El jugador puede iniciar la partida, controlar la dirección de la serpiente, ver el puntaje, reiniciar la partida y salir del juego.

El sistema, internamente, se encarga de mover la serpiente, generar comida, detectar si la serpiente comió, actualizar el puntaje, verificar colisiones y finalizar la partida.

Ver el diagrama realizado con Mermaid en ANEXOS A.

Explicación del diagrama de casos de uso

El jugador inicia la partida y el sistema prepara los elementos iniciales: serpiente, comida y puntaje. Durante la partida, el jugador controla la dirección de la serpiente mediante el teclado. El sistema interpreta esa entrada y actualiza el movimiento.

Mientras el juego se ejecuta, el sistema verifica constantemente si la serpiente comió la comida o si ocurrió una colisión. Si la serpiente come, aumenta su tamaño, se actualiza el puntaje y se genera una nueva comida. Si ocurre una colisión, la partida finaliza y se muestra el mensaje de Game Over junto con el puntaje final.

El jugador también puede reiniciar la partida después de perder o salir del juego.

7. Diagrama de flujo funcional

El diagrama de flujo funcional representa la lógica principal del juego desde que inicia hasta que termina.

Ver el diagrama realizado con Mermaid en ANEXOS B.

Explicación del diagrama de flujo

El programa inicia mostrando una pantalla inicial. Cuando el jugador decide comenzar, el sistema crea la serpiente, coloca el puntaje en cero y genera la primera comida.

Luego inicia el ciclo principal del juego. En este ciclo, el sistema lee la entrada del teclado, actualiza la dirección de la serpiente, mueve la serpiente y verifica si comió comida.

Si la serpiente comió, el sistema aumenta su tamaño, actualiza el puntaje y genera una nueva comida. Luego verifica si existe una colisión. Si no hay colisión, el ciclo continúa. Si hay colisión, el juego se detiene, se muestra Game Over y se presenta el puntaje final.

Finalmente, el jugador puede decidir si reinicia la partida o si termina el programa.

8. Diagrama de arquitectura

El diagrama de arquitectura muestra cómo estará organizado internamente el sistema. Para este proyecto se propone una arquitectura modular por componentes.

Ver el diagrama realizado con Mermaid en ANEXOS C.

También puede representarse como arquitectura por capas:

Ver el diagrama realizado con Mermaid en ANEXOS D.

9. Explicación de la arquitectura

La arquitectura propuesta para el Juego de la Serpiente se basa en una organización modular. Esto significa que el programa se divide en componentes, y cada componente tiene una responsabilidad específica.

Esta forma de organización permite que el sistema sea más claro, más fácil de desarrollar y más sencillo de modificar. Si en el futuro se desea agregar una nueva funcionalidad, como niveles de dificultad, sonidos, pausa o guardar el puntaje máximo, será más fácil hacerlo si el programa ya está separado en módulos.

9.1. Jugador

El jugador es el usuario que interactúa con el sistema. Su función principal es controlar la dirección de la serpiente utilizando el teclado. También puede iniciar la partida, reiniciar después de perder o salir del juego.

9.2. Interfaz gráfica

La interfaz gráfica es la parte visible del sistema. Muestra el tablero, la serpiente, la comida, el puntaje y los mensajes del juego.

Esta capa es importante porque permite que el usuario entienda lo que está ocurriendo durante la partida. La interfaz debe actualizarse constantemente para reflejar el movimiento de la serpiente y los cambios del juego.

9.3. Controlador principal del juego

El controlador principal coordina todo el funcionamiento del sistema. Se encarga de ejecutar el ciclo principal del juego, recibir las entradas del usuario, llamar a los módulos correspondientes y actualizar el estado general.

El controlador funciona como el centro del programa. No realiza todas las tareas directamente, sino que coordina a los demás módulos.

9.4. Módulo de entrada de teclado

Este módulo se encarga de detectar las teclas presionadas por el jugador. Su función es transformar la entrada del usuario en una dirección válida para la serpiente.

Por ejemplo, si el jugador presiona la flecha derecha, el módulo informa al sistema que la nueva dirección debe ser hacia la derecha.

También debe evitar cambios de dirección inválidos. Por ejemplo, si la serpiente se mueve hacia la derecha, no debería poder cambiar directamente hacia la izquierda.

9.5. Módulo de lógica de la serpiente

Este módulo administra el movimiento y crecimiento de la serpiente. Controla la posición de la cabeza y del cuerpo.

Cada vez que el juego avanza, este módulo calcula la nueva posición de la cabeza de la serpiente. Luego actualiza las posiciones del cuerpo para simular el movimiento.

Cuando la serpiente come, este módulo permite que el cuerpo crezca.

9.6. Módulo de comida

Este módulo se encarga de generar la comida dentro del tablero. La comida debe aparecer en una posición aleatoria válida.

El módulo debe asegurarse de que la comida no aparezca encima de la serpiente. Si la posición generada está ocupada, debe buscar otra posición.

9.7. Módulo de colisiones

Este módulo verifica si la serpiente ha chocado con una pared o consigo misma. La detección de colisiones es una de las partes más importantes del juego, ya que determina cuándo termina la partida.

También puede detectar si la serpiente alcanzó la comida, aunque esa validación también puede relacionarse con el módulo de comida.

9.8. Módulo de puntaje

Este módulo administra el puntaje del jugador. Al inicio de la partida, el puntaje es cero. Cada vez que la serpiente come, el puntaje aumenta.

El puntaje debe enviarse a la interfaz gráfica para que el jugador pueda visualizarlo en pantalla.

9.9. Módulo de estado del juego

Este módulo controla el estado actual de la partida. El juego puede estar en diferentes estados, como:

- Esperando inicio.
- En ejecución.
- Finalizado.
- Reiniciado.

Este módulo permite saber si el juego debe seguir moviendo la serpiente o si debe detenerse porque ocurrió una colisión.

9.10. Módulo de renderizado

El módulo de renderizado se encarga de dibujar o actualizar visualmente los elementos del juego en pantalla. Recibe información sobre la posición de la serpiente, la comida y el puntaje, y luego actualiza la interfaz gráfica.

10. Conclusión

El Juego de la Serpiente es una opción adecuada para esta actividad porque permite aplicar conceptos básicos e importantes del desarrollo de software. Aunque se trata de un juego sencillo, su diseño requiere analizar el problema, identificar funcionalidades, definir reglas, controlar entradas del usuario, gestionar estados y organizar el sistema en componentes.

A través del diagrama de casos de uso se identificaron las principales interacciones entre el jugador y el sistema. Este diagrama permitió comprender qué acciones puede realizar el usuario y qué funcionalidades debe ofrecer el juego.

Mediante el diagrama de flujo funcional se representó la lógica principal del programa. Este diagrama muestra el proceso desde el inicio de la partida hasta el fin del juego, incluyendo decisiones importantes como comer comida, actualizar puntaje, detectar colisiones y reiniciar.

Finalmente, el diagrama de arquitectura permitió representar la estructura interna del software. La propuesta modular divide el sistema en componentes como interfaz gráfica, controlador principal, entrada de teclado, lógica de la serpiente, comida, colisiones, puntaje, estado del juego y renderizado.

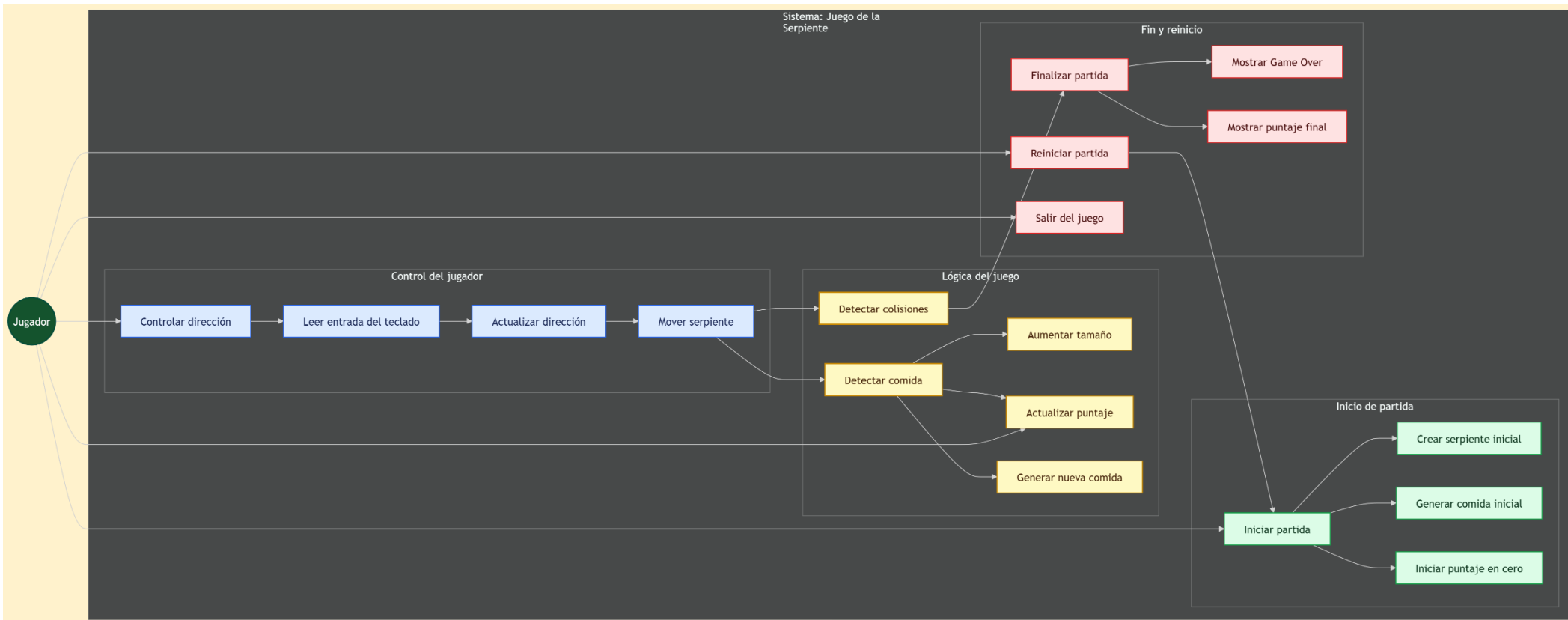
En conclusión, realizar estos diagramas antes de programar permite comprender mejor el sistema, evitar errores de diseño y facilitar el desarrollo posterior. La planificación previa es una etapa fundamental en la construcción de software, ya que transforma una idea general en una solución organizada, clara y lista para implementarse.

11. Bibliografía

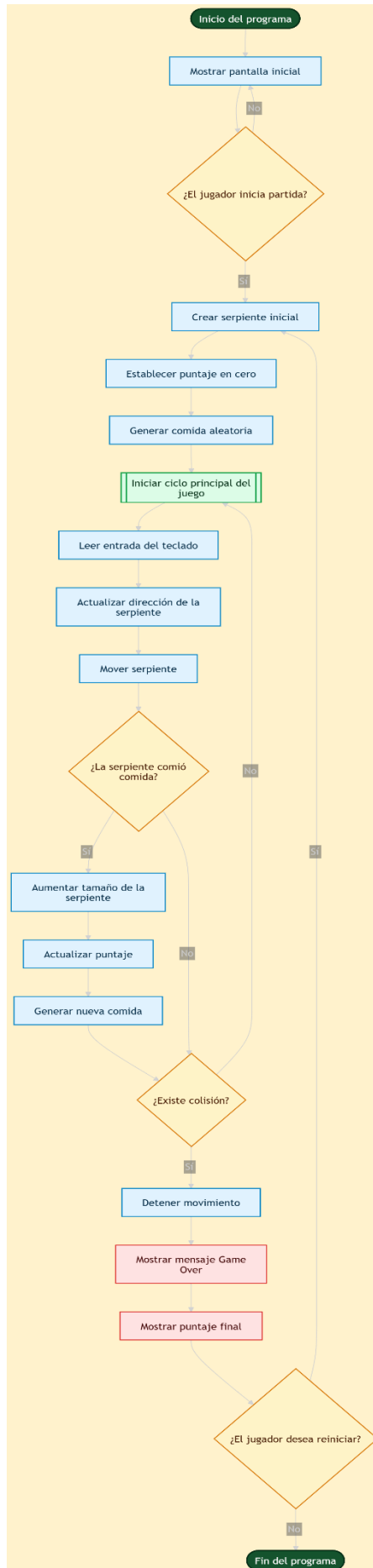
1. Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1/About-UML>
2. Lucidchart. (s. f.). *Flowchart symbols and notation*. Recuperado el 31 de mayo de 2026, de <https://www.lucidchart.com/pages/flowchart-symbols-meaning-explained>
3. IEEE Standards Association. (2011). *ISO/IEC/IEEE 42010:2011: Systems and software engineering—Architecture description*. <https://standards.ieee.org/standard/42010-2011.html>
4. Brown, S. (s. f.). *The C4 model for visualising software architecture*. Recuperado el 31 de mayo de 2026, de <https://c4model.com/>

ANEXOS

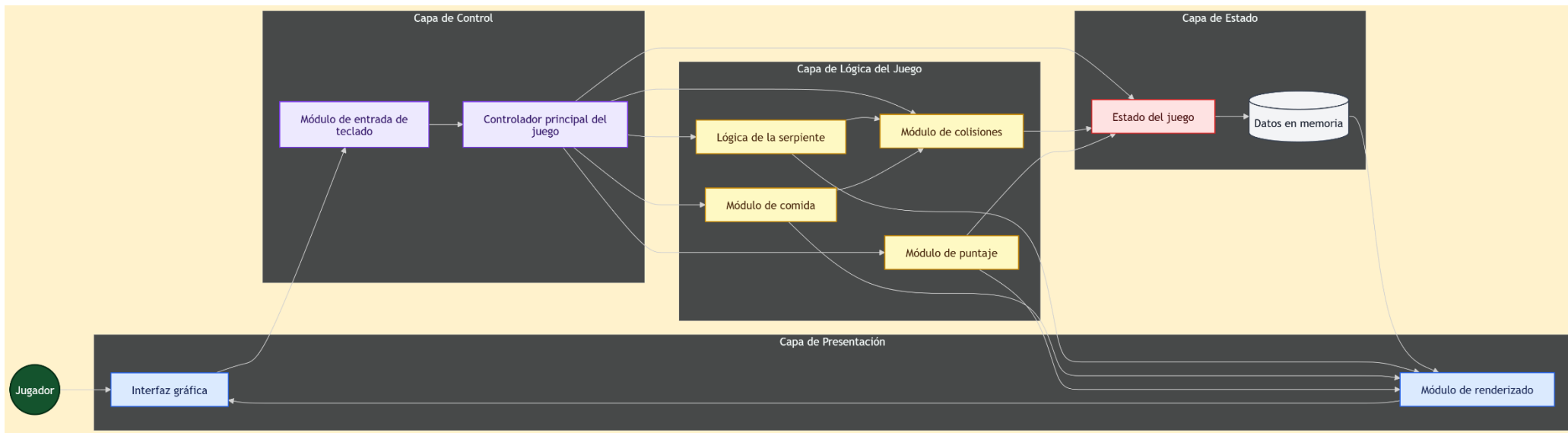
A. Diagrama funcional tipo casos de uso



B. Diagrama de flujo funcional



C. Diagrama de arquitectura por componentes



D. Diagrama de arquitectura por capas

